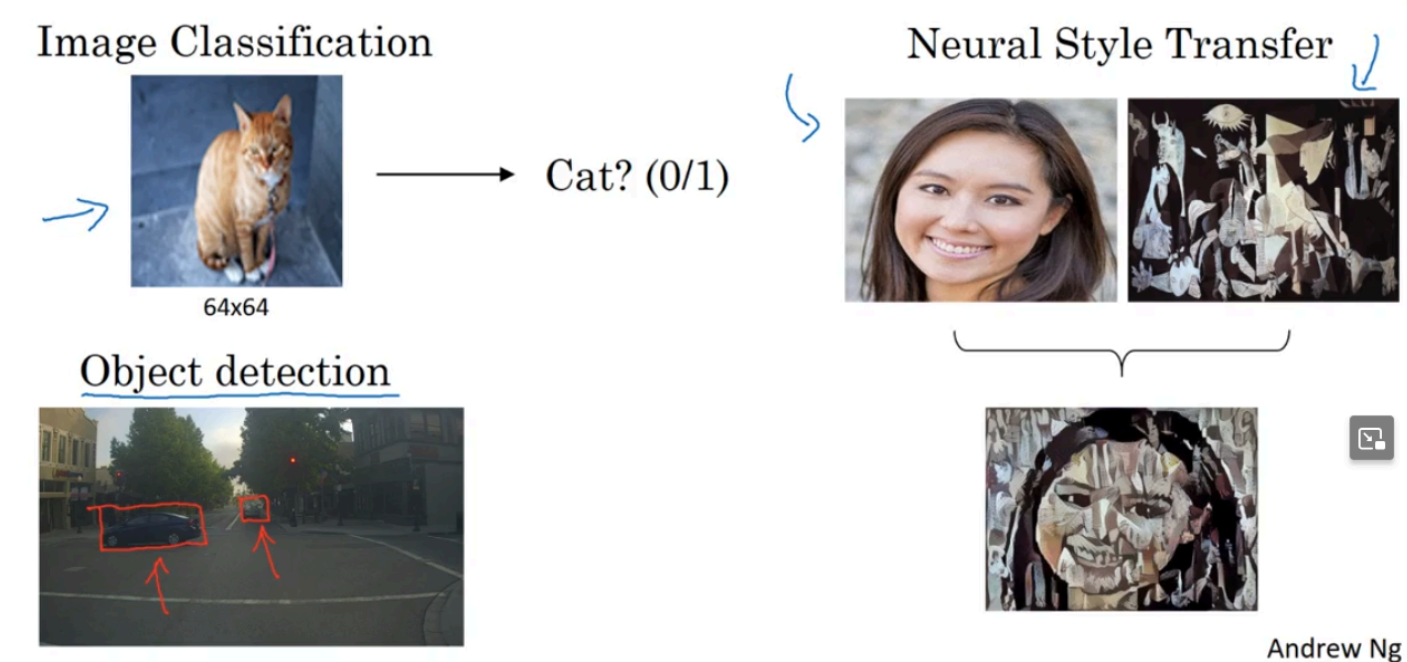


Foundations of Convolutional Neural Networks

Computer Vision

Computer Vision Problems



Here's a clean, concise summary you can put in your notebook:

Convolutional Neural Networks (CNNs) – Introduction

- **Deep learning in computer vision** has enabled major breakthroughs:
 - Self-driving cars detect objects (cars, pedestrians)
 - Face recognition (unlock phones, security)
 - Image recommendation (food, hotels, scenery)
 - AI-generated art (e.g., neural style transfer)

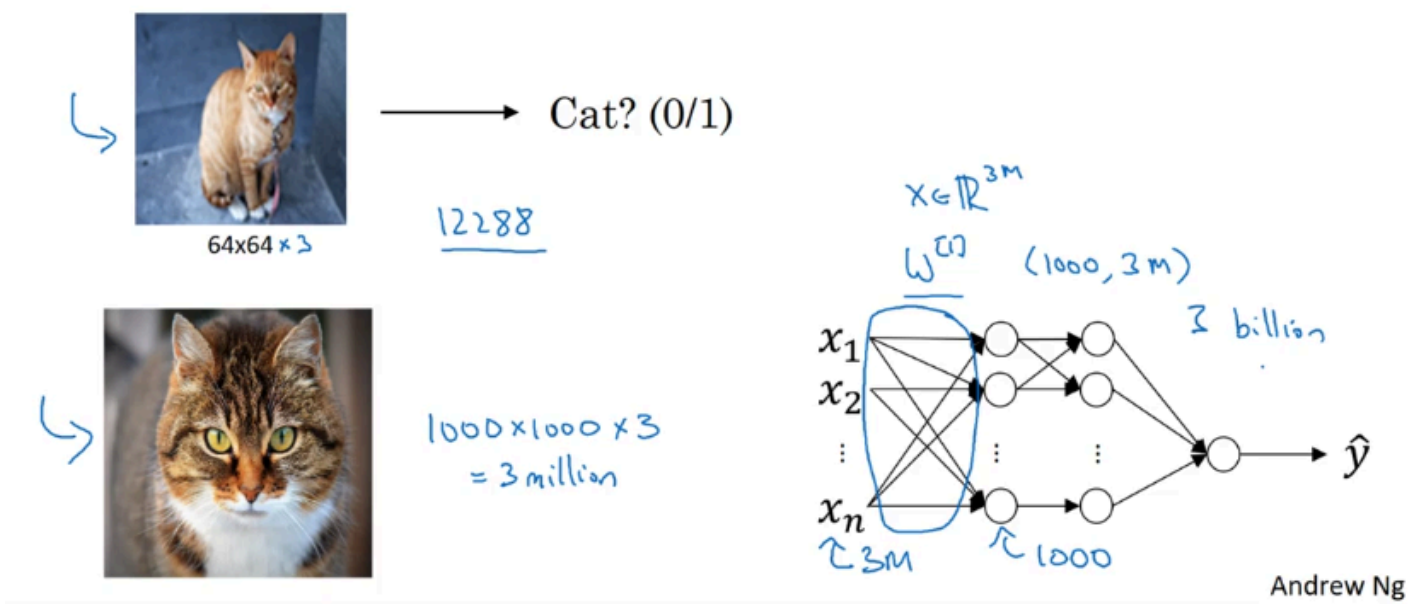
Why CNNs are important

1. **Enable new applications** that were impossible before
2. **Inspire other fields** (ideas transfer to areas like speech recognition)

Key Computer Vision Tasks

- **Image Classification:** Identify what is in an image (e.g., cat vs. non-cat)
- **Object Detection:** Detect objects *and their locations* (draw bounding boxes)
- **Neural Style Transfer:** Combine content + artistic style to create new images

Deep Learning on large images



Challenge in Computer Vision

- Images have **very high dimensional input**
 - Example:
 - $64 \times 64 \times 3 \rightarrow 12,288$ features (manageable)
 - $1000 \times 1000 \times 3 \rightarrow 3,000,000$ features (huge!)
- Fully connected networks become impractical:
 - Too many parameters (billions)
 - High computation & memory cost
 - High risk of **overfitting**

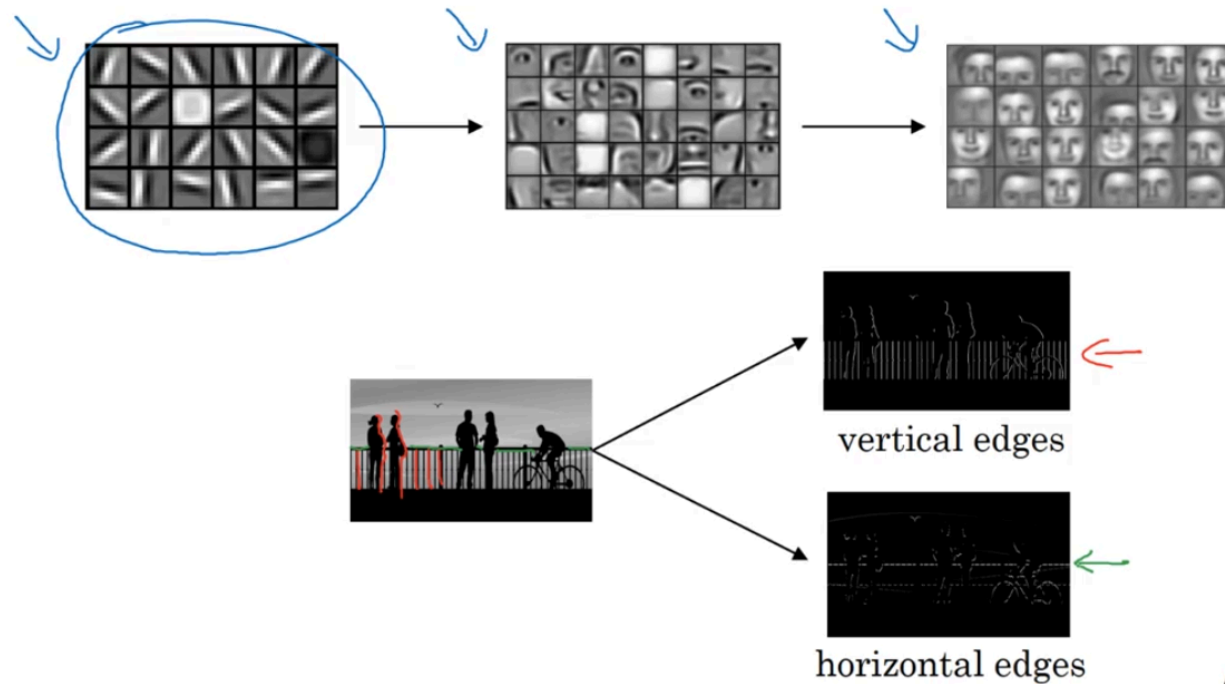
Solution: Convolutional Neural Networks (CNNs)

- Use **convolution operations** to handle large images efficiently
- Reduce parameters while preserving important spatial features
- Fundamental for modern computer vision systems

Next concept: **Convolution operation (e.g., edge detection)**

Edge Detection Example

Computer Vision Problem



Andrew Ng

Convolution Operation & Edge Detection

- **Convolution** is a **core building block of CNNs**
- It helps detect **low-level features** like edges, which are used by deeper layers to detect:
 - Shapes → Parts of objects → Full objects (e.g., faces)

Idea: Edge Detection

- Before recognizing objects, a model first finds **edges** in an image
- Edges highlight important structures:
 - **Vertical edges** → buildings, people outlines
 - **Horizontal edges** → railings, horizons

How it works (intuitively)

- Apply a small filter (kernel) that **slides over the image**
- The filter **detects changes** in pixel values:
 - Strong change → edge detected
 - Small change → no edge

Types of Edges

- **Vertical edge detector** → highlights vertical lines
- **Horizontal edge detector** → highlights horizontal lines

Key Insight

- Convolution transforms the image into **feature maps** that highlight useful patterns (like edges)
- These features are the foundation for higher-level understanding in CNNs

Next: Learn specific filters (like Sobel filters) and how convolution is computed mathematically

Vertical edge detection

$$3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 5 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6x6

"convolution"

*

1	0	-1
1	0	-1
1	0	-1

3x3
filter

=

-5	-4	0	8

4x4

Andrew Ng

Compute a convolution (like image above)

Vertical edge detection

$$3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 5 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6x6

"convolution"

*

1	0	-1
1	0	-1
1	0	-1

3x3
filter

=

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

4x4

python: conv-forward
tensorflow: tf.nn.conv2d
keras: Conv2D

Andrew Ng

Here's a clear, structured summary for your notebook:

Vertical edge detection

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

6x6

*

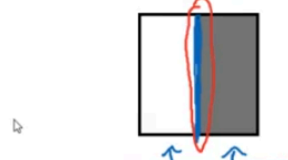
1	0	-1
1	0	-1
1	0	-1

3x3

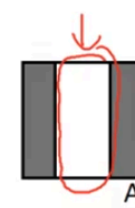
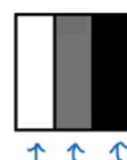
=

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

4x4



*



Andrew Ng

Why Convolution Detects Vertical Edges

- Consider a simple **6×6 image**:
 - Left half = bright pixels (value 10)
 - Right half = dark pixels (value 0)
 - This creates a **strong vertical edge** in the middle
-

The 3×3 Vertical Edge Filter

- The filter looks like:

```
[ 1  0 -1
  1  0 -1
  1  0 -1 ]
```

- Interpretation:
 - Left column → positive (detect bright region)
 - Middle → ignore
 - Right column → negative (detect dark region)
-

How Convolution Detects the Edge

- The filter slides over the image and computes:
 - **(left side values) – (right side values)**
 - Results:
 - If left $\approx$ right → output $\approx$ 0 (no edge)
 - If left $\gg$ right → large positive value → **vertical edge detected**
 - If left $\ll$ right → large negative value → edge in opposite direction
-

Key Observation

- At the boundary (bright → dark):
 - Example: $(10 + 10 + 10) - (0 + 0 + 0) = \mathbf{30}$
 - This gives a **strong activation** → highlights the edge
-

Intuition

- A **vertical edge** =
- 🖐️ bright pixels on the left + dark pixels on the right

$$\begin{array}{c} \downarrow \\ * \end{array} \begin{array}{|c|c|c|} \hline 1 & 0 & -1 \\ \hline 1 & 0 & -1 \\ \hline 1 & 0 & -1 \\ \hline \end{array} \begin{array}{c} \\ \\ 3 \times 3 \end{array}$$

- Convolution captures this pattern using simple weighted sums

Note

- In small images (like 6×6), edges may look thick
- In real images (e.g., 1000×1000), edges are detected much more precisely

Key Takeaway

- Convolution lets us **encode patterns (like edges) using filters**
- These detected edges become the **foundation for higher-level features in CNNs**

👉 Next: Using convolution as a layer in a full CNN architecture

More Edge Detection

Vertical edge detection examples

$$\begin{array}{|c|c|c|c|c|c|} \hline 10 & 10 & 10 & 0 & 0 & 0 \\ \hline 10 & 10 & 10 & 0 & 0 & 0 \\ \hline 10 & 10 & 10 & 0 & 0 & 0 \\ \hline 10 & 10 & 10 & 0 & 0 & 0 \\ \hline 10 & 10 & 10 & 0 & 0 & 0 \\ \hline 10 & 10 & 10 & 0 & 0 & 0 \\ \hline \end{array} * \begin{array}{|c|c|c|} \hline 1 & 0 & -1 \\ \hline 1 & 0 & -1 \\ \hline 1 & 0 & -1 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline 0 & 30 & 30 & 0 \\ \hline 0 & 30 & 30 & 0 \\ \hline 0 & 30 & 30 & 0 \\ \hline 0 & 30 & 30 & 0 \\ \hline \end{array}$$

→ 

$$\begin{array}{|c|c|c|c|c|c|} \hline 0 & 0 & 0 & 10 & 10 & 10 \\ \hline 0 & 0 & 0 & 10 & 10 & 10 \\ \hline 0 & 0 & 0 & 10 & 10 & 10 \\ \hline 0 & 0 & 0 & 10 & 10 & 10 \\ \hline 0 & 0 & 0 & 10 & 10 & 10 \\ \hline 0 & 0 & 0 & 10 & 10 & 10 \\ \hline \end{array} * \begin{array}{|c|c|c|} \hline 1 & 0 & -1 \\ \hline 1 & 0 & -1 \\ \hline 1 & 0 & -1 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline 0 & -30 & -30 & 0 \\ \hline 0 & -30 & -30 & 0 \\ \hline 0 & -30 & -30 & 0 \\ \hline 0 & -30 & -30 & 0 \\ \hline \end{array}$$

→ 

And if you don't care which

Andrew Ng

Vertical Edge Detection: Positive vs. Negative Edges

Key Points

- **Direction of Transition:** Vertical edge detection distinguishes between different types of light transitions:
 - **Light to Dark:** Results in positive values (e.g., \$+30\$) in the output matrix.
 - **Dark to Light:** Results in negative values (e.g., \$-30\$) in the output matrix.

Edge Detection: From Hand-Coded Filters to Learned Parameters

Key Points

- **Historical Context:** Computer vision researchers debated for years over the "best" numbers to use in a 3×3 filter for edge detection.
- **Hand-Coded Filters:**
 - **Sobel Filter:** Uses weights like `[1, 2, 1]` in the first column and `[-1, -2, -1]` in the third. The middle row gets more weight, making the filter more **robust**.
 - **Scharr Filter:** Uses weights like `[3, 10, 3]` and `[-3, -10, -3]`, offering slightly different detection properties.
- **Orientation:** Filters can be rotated (e.g., 90 degrees) to switch from **vertical** to **horizontal** edge detection.
- **The Deep Learning Shift:** Instead of humans hand-picking these nine numbers, modern algorithms treat them as **parameters** to be learned via **backpropagation**.
- **Advantages of Learning Filters:**
 - **Flexibility:** The network can learn to detect edges at any orientation (e.g., 45°, 70°, or 73°).
 - **Data-Driven:** The network can discover filters that capture data statistics more effectively than any hand-coded version.
 - **Robustness:** Learned features are often more robust than those manually designed by researchers.
- **Consistency:** Regardless of whether the filter is hand-coded or learned, the underlying operation remains **convolution**, applied repeatedly across the entire image.

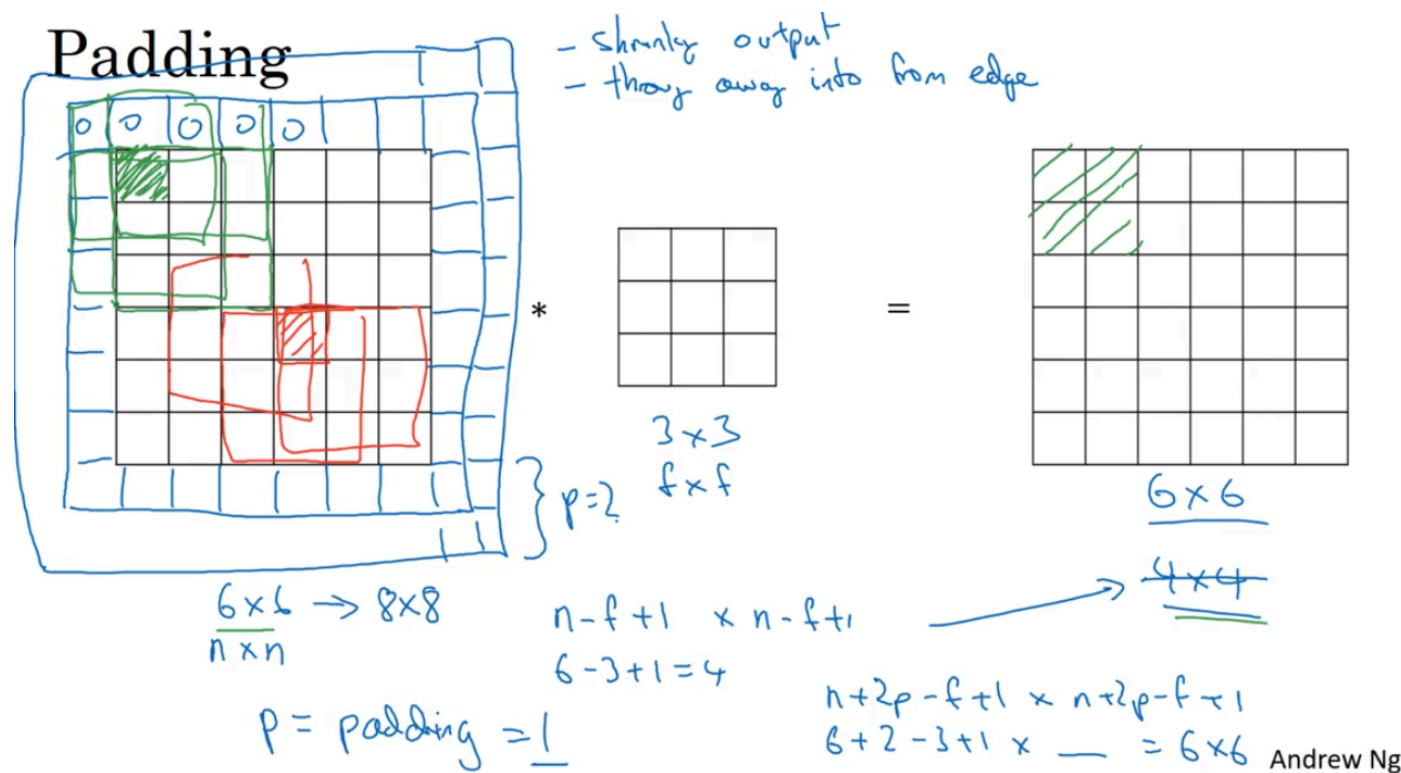
Important Terms / Concepts

- **Sobel Filter:** A specific 3 × 3 filter that prioritizes the central pixel for better stability.
- **Scharr Filter:** A hand-coded filter variant with higher weights on the central row.
- **Backpropagation:** The process used to automatically update the nine numbers (parameters) in a filter based on data.
- **Parameters:** The specific values within a filter matrix that determine what feature (edge, orientation, etc.) the filter will detect.

Summary

While researchers traditionally used hand-coded filters like Sobel and Scharr to detect vertical and horizontal edges, deep learning allows these filters to be treated as learnable parameters. By using backpropagation, a neural network can automatically determine the optimal values for a 3×3 matrix to detect complex edges at any orientation. This data-driven approach results in more robust feature detection than manual engineering.

Padding



Padding in Convolutional Neural Networks

Padding is a critical modification to the basic convolution operation, essential for building deep neural networks. It involves adding a border of pixels (usually zeros) around the edge of an image before applying a filter.

The Two Problems with Basic Convolution

Without padding, two major issues arise every time a convolution is applied:

1. **Shrinking Output:** The image size decreases with every layer. In a very deep network (e.g., 100 layers), the image would eventually shrink to 1×1 , limiting how many layers you can use.
2. **Loss of Edge Information:** Pixels at the corners or edges are used in very few filter positions compared to central pixels. This effectively "throws away" information from the image boundaries.

Key Points: The Math of Padding

- **Standard Convolution Formula:** For an $n \times n$ image and an $f \times f$ filter, the output is:

$$(n - f + 1) \times (n - f + 1)$$

- *Example:* A 6×6 image with a 3×3 filter results in a 4×4 output.

- **Convolution with Padding (p):** If you add p pixels of padding to all sides, the new formula is:

$$(n + 2p - f + 1) \times (n + 2p - f + 1)$$

- *Example:* Padding a 6×6 image with $p = 1$ creates an 8×8 temporary image. Convoluting this with a 3×3 filter results in a 6×6 output ($8 - 3 + 1 = 6$), **preserving the original size**.

Important Terms / Concepts

- **Padding (p):** The number of pixel layers added to the border of the image. By convention, these are filled with **zeros**.
- **n :** The dimension of the input image ($n \times n$).
- **f :** The dimension of the filter/kernel ($f \times f$).
- **Border Preservation:** Using padding allows edge pixels to influence more cells in the output, reducing the imbalance between edge and center information.

Summary

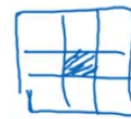
Padding solves the issues of shrinking image dimensions and the loss of boundary information by adding extra pixels around the input image. This allows researchers to build much deeper neural networks while ensuring that pixels at the edges of the image are weighted more heavily in the resulting feature maps.

Valid and Same convolutions

Valid: $n \times n$ $\times$ $f \times f \rightarrow \frac{n-f+1}{1} \times \frac{n-f+1}{1}$
 6×6 $\times$ $3 \times 3 \rightarrow 4 \times 4$

Same: Pad so that output size is the same as the input size.

$n+2p-f+1 \times n+2p-f+1$
 $n+2p-f+1 = n \Rightarrow p = \frac{f-1}{2}$
 3×3 $p = \frac{3-1}{2} = 1$ | 5×5 $p=2$



Andrew Ng

- Valid convolution: Convolution with no padding
- Same convolution: Pad so that output size is the same as the input size ($p = \frac{f-1}{2}$)
 - So that, the f is usually an odd number

Strided Convolutions

Strided convolution

$$\begin{bmatrix} 2 & 3 & 7 & 4 & 6 & 2 & 9 \\ 6 & 6 & 9 & 8 & 7 & 4 & 3 \\ 3 & 4 & 8 & 3 & 8 & 9 & 7 \\ 7 & 8 & 3 & 6 & 6 & 3 & 4 \\ 4 & 2 & 1 & 8 & 3 & 4 & 6 \\ 3 & 2 & 4 & 1 & 9 & 8 & 3 \\ 0 & 1 & 3 & 9 & 2 & 1 & 4 \end{bmatrix} \times \begin{bmatrix} 3 & 4 & 4 \\ 1 & 0 & 2 \\ -1 & 0 & 3 \end{bmatrix} = \begin{bmatrix} 91 & 100 & 83 \\ 69 & 91 & 127 \\ 44 & 72 & 74 \end{bmatrix}$$

7×7 $\times$ 3×3
 padding p $\times$ $f \times f$
 stride s
 $s=2$

$\left\lfloor \frac{7+0-3}{2} + 1 \right\rfloor \times \left\lfloor \frac{7+0-3}{2} + 1 \right\rfloor$
 $\frac{7+0-3}{2} + 1 = \frac{4}{2} + 1 = 3$

$\lfloor z \rfloor = \text{floor}(z)$

Andrew Ng

Stride Convolutions: Controlling the Step Size

Stride is a parameter that determines how many pixels the filter "jumps" as it moves across the input image. It is a powerful tool for reducing the spatial dimensions of your data.

Key Concepts

- Definition of Stride (s):** The number of steps the filter moves horizontally or vertically.
 - Stride = 1:** The filter moves one pixel at a time (standard convolution).
 - Stride = 2:** The filter skips one position, moving two pixels at a time.
- The Convolution Rule:** The filter must lie **entirely within the image** (or the image plus padding) to perform a computation. If a stride would cause the filter to "hang" outside the boundaries, that specific calculation is skipped.
- Rounding Convention:** If the math for the output size results in a fraction, always **round down** (take the "floor").

The General Formula

To calculate the output dimension for an $n \times n$ image, $f \times f$ filter, padding p , and stride s :

$$\text{Output Size} = \left\lfloor \frac{n + 2p - f}{s} \right\rfloor + 1$$

Example from the script:

- Input (n): 7
- Filter (f): 3
- Padding (p): 0
- Stride (s): 2

Calculation: $\frac{7 + 0 - 3}{2} + 1 = \frac{4}{2} + 1 = 3$

Result: A 3×3 output matrix.

Important Terms

- **Stride (s):** The step size of the filter.
- **Floor Notation ($\lfloor z \rfloor$):** The mathematical operation of rounding a number z down to the nearest integer.
- **Element-wise Product & Sum:** The standard process of multiplying filter values by image pixels and adding them up to get one output value.

Summary

Stride convolutions allow you to skip pixels during the convolution process, effectively downsampling the image as you detect features. By using the floor of the dimension formula, we ensure the filter only operates on valid regions where it fits entirely within the input boundaries.

Summary of convolutions

$n \times n$ image $f \times f$ filter
padding p stride s

Output Size:

$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \underbrace{\frac{n+2p-f}{s}} + 1 \right\rfloor$$



Andrew Ng

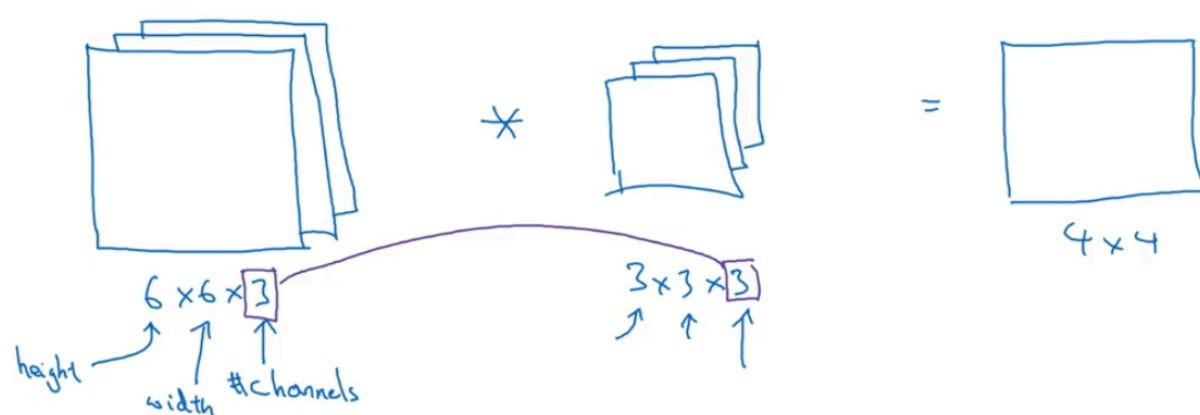
- **Convention:** The standard practice in a specific field (in this case, using the word "convolution" to mean "cross-correlation").

Summary

While traditional mathematics defines convolution as requiring a horizontal and vertical flip of the filter, deep learning skips this step for simplicity. This "unflipped" version is technically called cross-correlation, but it is referred to as convolution throughout machine learning literature. This distinction does not change how you implement or understand neural networks, as the model learns the necessary filter weights regardless.

Convolutions Over Volume

Convolutions on RGB images



Let's go through in detail how this works but let's use a more

The shift from 2D to 3D convolutions is a major step in processing real-world data like RGB (color) images. The key takeaway here is that **depth must match** between the image and the filter.

Convolutions Over 3D Volumes

When dealing with color images, we no longer work with a single grid of pixels. Instead, we work with a **volume** that has height, width, and depth (channels).

1. Dimensionality Rules

To convolve a color image, your filter must become three-dimensional as well.

- **Input Image:** $6 \times 6 \times 3$ (Height x Width x **Channels**)
- **Filter (Kernel):** $3 \times 3 \times 3$ (Height x Width x **Channels**)
- **The Golden Rule:** The number of channels in the input **must exactly match** the number of channels in the filter.

2. Understanding the Output

Even though the input and the filter are 3D volumes, the output of a single filter convolution is a **2D matrix**.

- **Output Size:** 4×4 (or $4 \times 4 \times 1$).
- **Why?** The convolution operation sums up all the products across all three channels (Red, Green, and Blue) into a single real number for each position.

Important Terms / Concepts

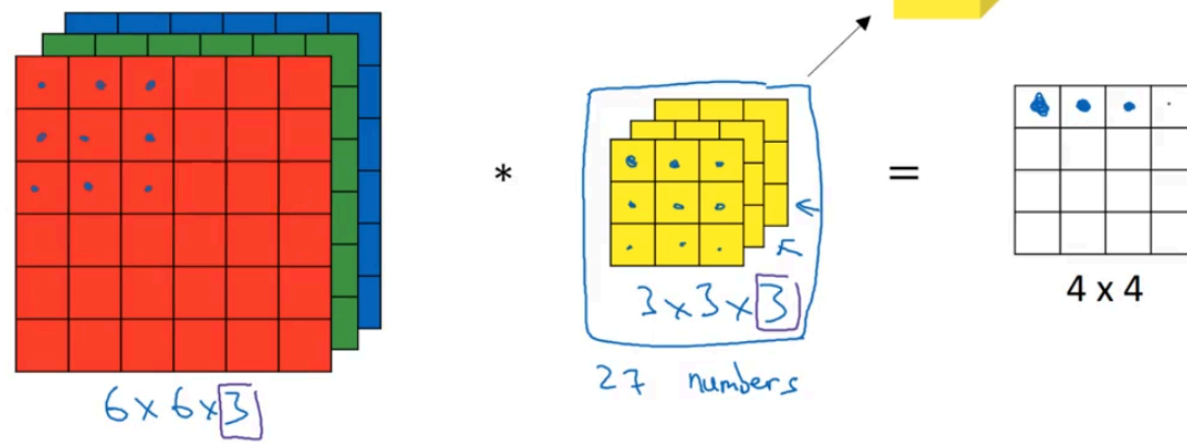
- **Channels:** The "depth" of the image. For RGB, there are 3 channels (Red, Green, Blue).
- **3D Filter:** A stack of weights that matches the depth of the input. Each "slice" of the filter looks for features in its corresponding color channel.

- **Channel Matching:** The requirement that the third dimension of the filter (e.g., 3) must equal the third dimension of the input (e.g., 3).

Summary

In 3D convolutions, an RGB image is treated as a stack of three color channels, requiring a filter that also has three layers of depth. The convolution process collapses these three dimensions into a single 2D output map, provided that the number of channels in the filter matches the number of channels in the input.

Convolutions on RGB image



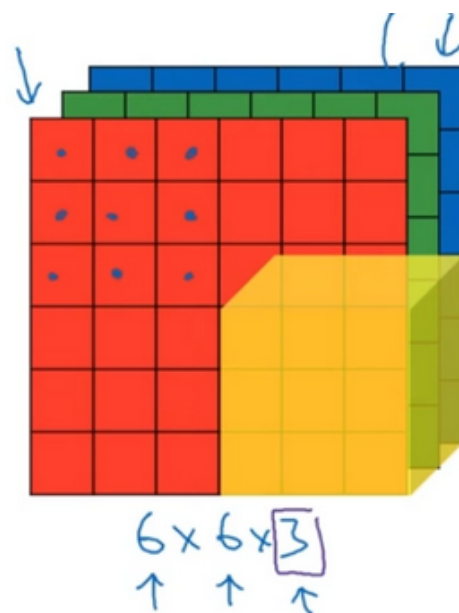
Andrew Ng

This lesson explains how convolution transitions from 2D grayscale images to 3D color volumes (RGB). The core mechanism relies on a 3D filter that processes all color channels simultaneously to produce a single feature map.

2. The Convolution Operation on Volumes

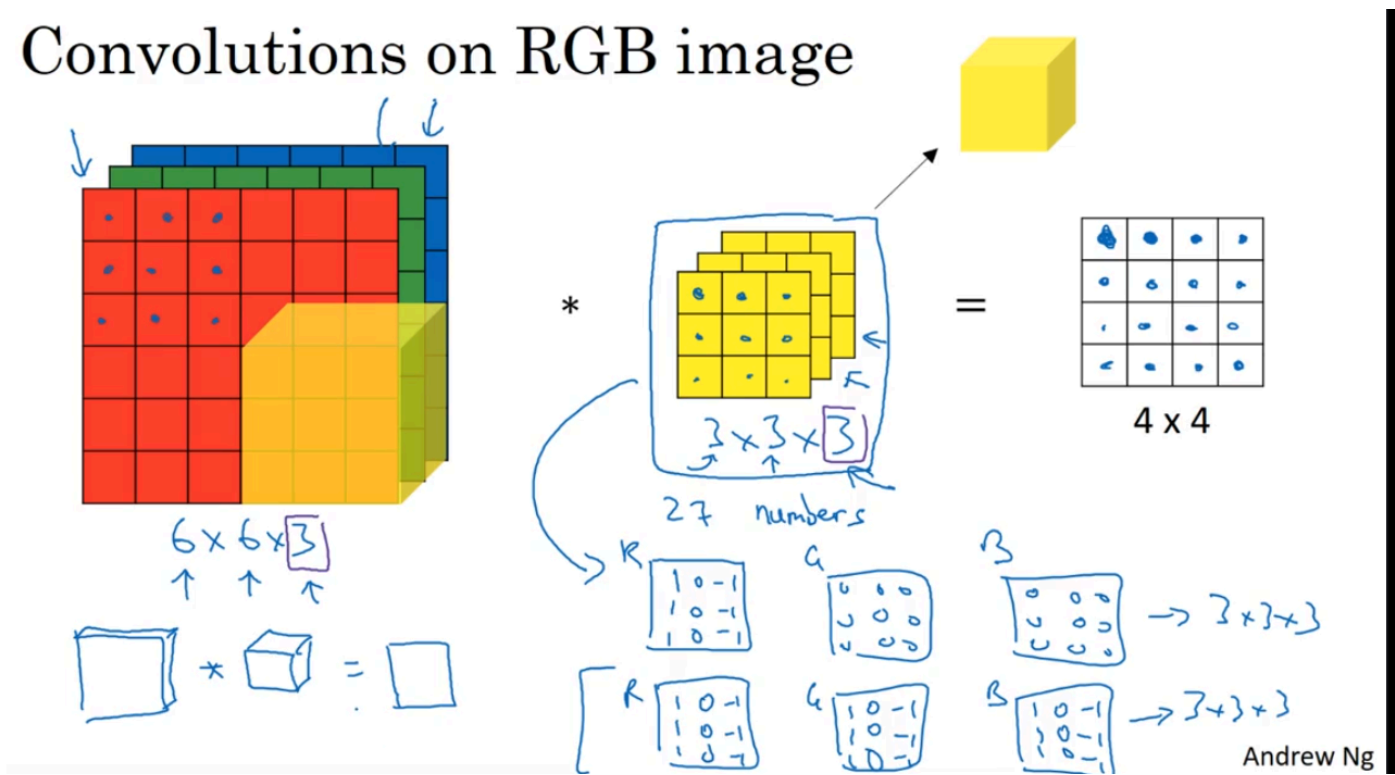
The process involves sliding the 3D filter cube through the 3D image volume.

- **Parameters:** A $3 \times 3 \times 3$ filter contains **27 numbers** (parameters).
- **Calculation:**
 1. Place the filter at a position in the image.
 2. Perform **27 multiplications** (each filter parameter multiplied by its corresponding pixel in the Red, Green, and Blue channels).



1. **Sum all 27 results** into a single real number.
- **Output:** Despite 3D inputs, the result of a single filter is a **2D matrix** (e.g., 4×4).

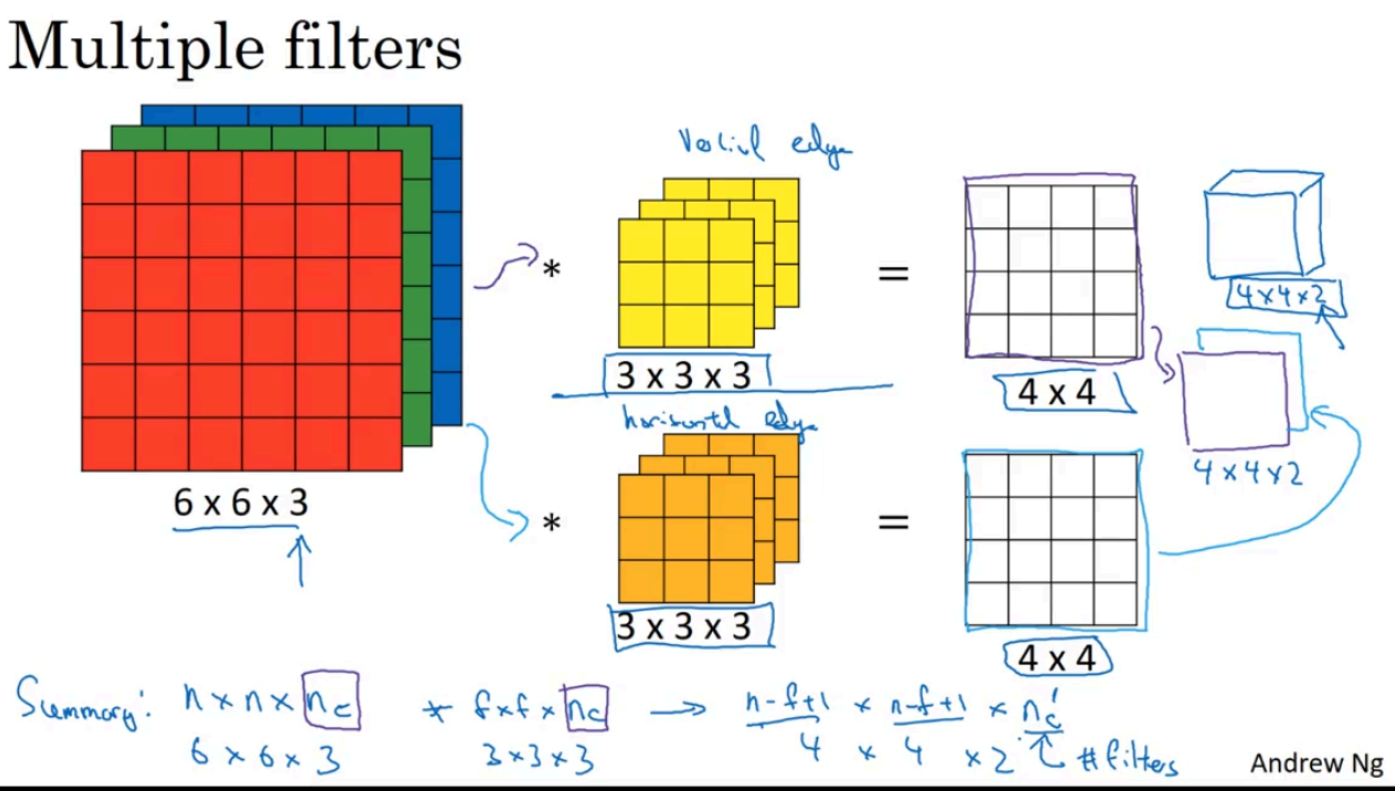
3. Channel-Specific Feature Detection



Because the filter has 27 independent parameters, you can design it to look for very specific color-based features:

Goal	Filter Configuration
Detect Red Edges Only	Red channel has edge weights (1, 0, -1); Green/Blue channels are all zeros .
Detect Edges in Any Color	All three channels (R, G, B) have the same edge detection weights.
General Features	Different weights in each channel to detect specific color/texture combinations.

Key Takeaway: By treating the 27 numbers as parameters, the network can learn to detect edges at any orientation, in any specific color channel, or across all channels simultaneously.



This lesson introduces one of the most important concepts in Convolutional Neural Networks (CNNs): how multiple filters work together to transform an input volume into an output volume with multiple channels.

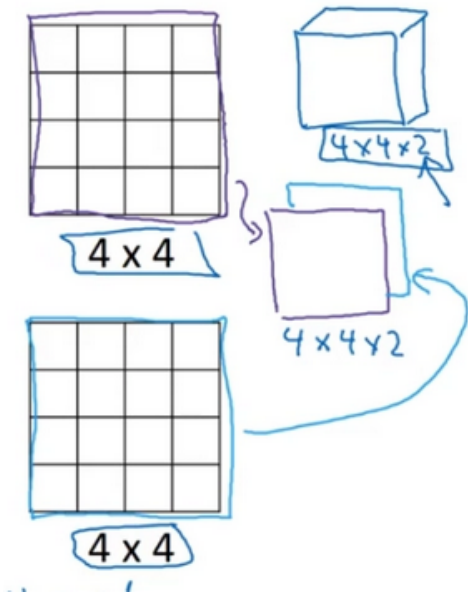
Using Multiple Filters Simultaneously

Detecting just one type of feature (like a vertical edge) isn't enough for complex image recognition. We need to detect many features at once — horizontal edges, 45-degree angles, or specific color textures.

1. The Stacking Mechanism

When you apply multiple filters to a single input, each filter produces its own 2D output. These outputs are then stacked along the third dimension.

- **Filter 1 (e.g., Vertical):** Processes the $6 \times 6 \times 3$ input → Results in a 4×4 output.
- **Filter 2 (e.g., Horizontal):** Processes the same $6 \times 6 \times 3$ input → Results in a different 4×4 output.
- **The Result:** By stacking these, you get a $4 \times 4 \times 2$ volume.



2. Dimension Summary Formula

The dimensions of your output are determined by your input size, your filter size, and the **number of filters** you choose to use.

Component	Dimensions	Note
Input Image	$n \times n \times n_c$	n_c = Number of channels (e.g., 3 for RGB)
Filter(s)	$f \times f \times n_c$	Depth n_c must match the input depth
Output Volume	$[n-f+1] \times [n-f+1] \times n_{c'}$	$n_{c'}$ = Number of filters used

Key Insight: The "3" in the input corresponds to the RGB channels, but the "2" (or any number $n_{c'}$) in the output corresponds to the **number of distinct features** you are detecting.

Important Terms / Concepts

- **n_c (Number of Channels):** Refers to the third dimension of the volume.
- **Depth:** A common synonym for "Channels" in computer vision literature, though "Channels" is often preferred to avoid confusion with the "depth" (number of layers) of the neural network itself.
- **Feature Map:** Each 2D "slice" in the output volume represents a specific feature detected across the image.
- **Volume Convolution:** The process of transforming a 3D input into a 3D output using a bank of 3D filters.

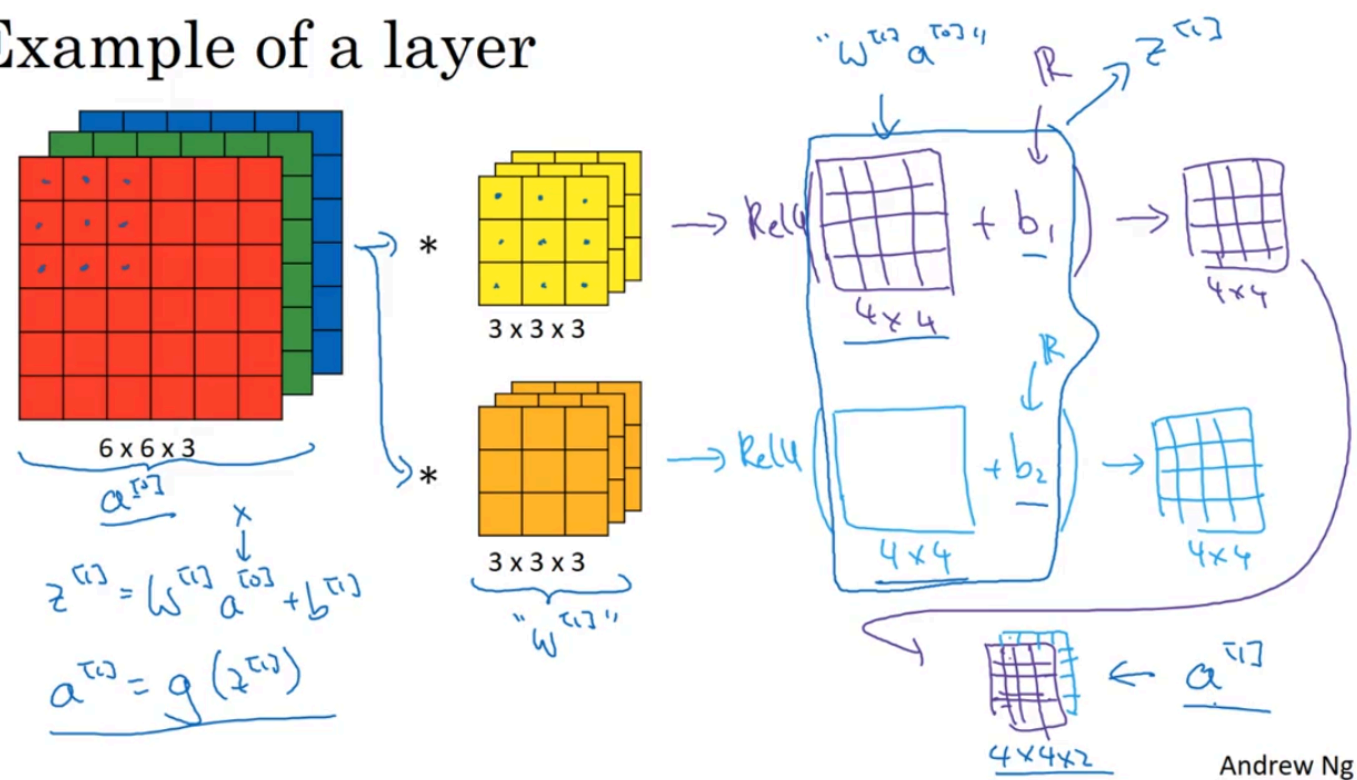
Summary

By using multiple filters at once, a single convolution layer can detect dozens or even hundreds of different features (vertical edges, horizontal edges, etc.) simultaneously. The output of this operation is a new 3D volume where the height and width are determined by the convolution math, and the number of channels is exactly equal to the number of filters applied.

One layer of a Convolutional Network

Example of a layer

Example of a layer



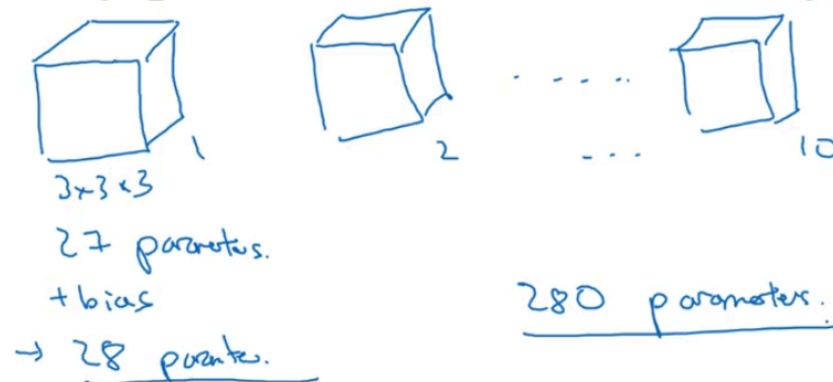
Andrew Ng

we can think it's like a normal linear feed forward 🤔

Exercise time!

Number of parameters in one layer

If you have 10 filters that are $3 \times 3 \times 3$ in one layer of a neural network, how many parameters does that layer have?



Andrew Ng

Calculating parameters is a foundational skill in deep learning. This script highlights why Convolutional Neural Networks (CNNs) are so efficient: the number of parameters depends on the **filters**, not the size of the input image.

Parameter Calculation in a Conv Layer

To find the total number of parameters in a single layer, you follow a simple three-step logic.

1. Parameters per Filter

Each 3D filter is a "cube" of numbers plus one extra number for the bias (\$b\$).

- **Weights:** $f \times f \times n_c$ (Filter Height x Filter Width x Input Channels)
- **Bias:** +1 (Each filter has exactly one bias term)
- **Total per filter:** $f \times f \times n_c + 1$

Example from script: A $3 \times 3 \times 3$ filter has 27 weights + 1 bias = **28 parameters**.

2. Total Layer Parameters

To get the total for the entire layer, multiply the parameters per filter by the number of filters you are using.

- **Formula:** $\text{Total} = \text{Filters} \times [(f \times f \times n_c) + 1(\text{bias})]$
- **Example from script:** $10 \text{ filters} \times 28 \text{ parameters} = \mathbf{280} \text{ parameters}$

Key Insight: Parameter Sharing

The most powerful property mentioned in the script is that **parameters are independent of image size**.

Input Image Size	Filter Configuration	Total Parameters
$6 \times 6 \times 3$	10 filters ($3 \times 3 \times 3$)	280
$1000 \times 1000 \times 3$	10 filters ($3 \times 3 \times 3$)	280
$5000 \times 5000 \times 3$	10 filters ($3 \times 3 \times 3$)	280

Why this matters:

- **Reduced Overfitting:** Because the number of parameters is small and fixed, the model is less likely to "memorize" the training data.
- **Efficiency:** You can learn features (like edges) on small images and apply those same 280 parameters to massive images without increasing the model's memory footprint.

Important Terms / Concepts

- **Bias (b):** The single constant added to the output of each filter's convolution.
- **Fixed Parameters:** The property where model complexity is determined by filter choice, not input resolution.
- **Overfitting:** A common AI problem where a model becomes too complex; CNNs fight this by keeping parameter counts low through shared filters.

Summary of notation

If layer l is a convolution layer:

$f^{[l]}$ = filter size
 $p^{[l]}$ = padding
 $s^{[l]}$ = stride
 $n_c^{[l]}$ = number of filters
 → Each filter is: $f^{[l-1]} \times f^{[l-1]} \times n_c^{[l-1]}$
 Activations: $A^{[l]} \rightarrow n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$
 Weights: $f^{[l-1]} \times f^{[l-1]} \times n_c^{[l-1]} \times n_c^{[l]}$
 bias: $n_c^{[l]} - (1, 1, 1, n_c^{[l]})$ ← #filters in layer l.

Input: $n_H^{[l-1]} \times n_W^{[l-1]} \times n_c^{[l-1]}$
 Output: $n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$

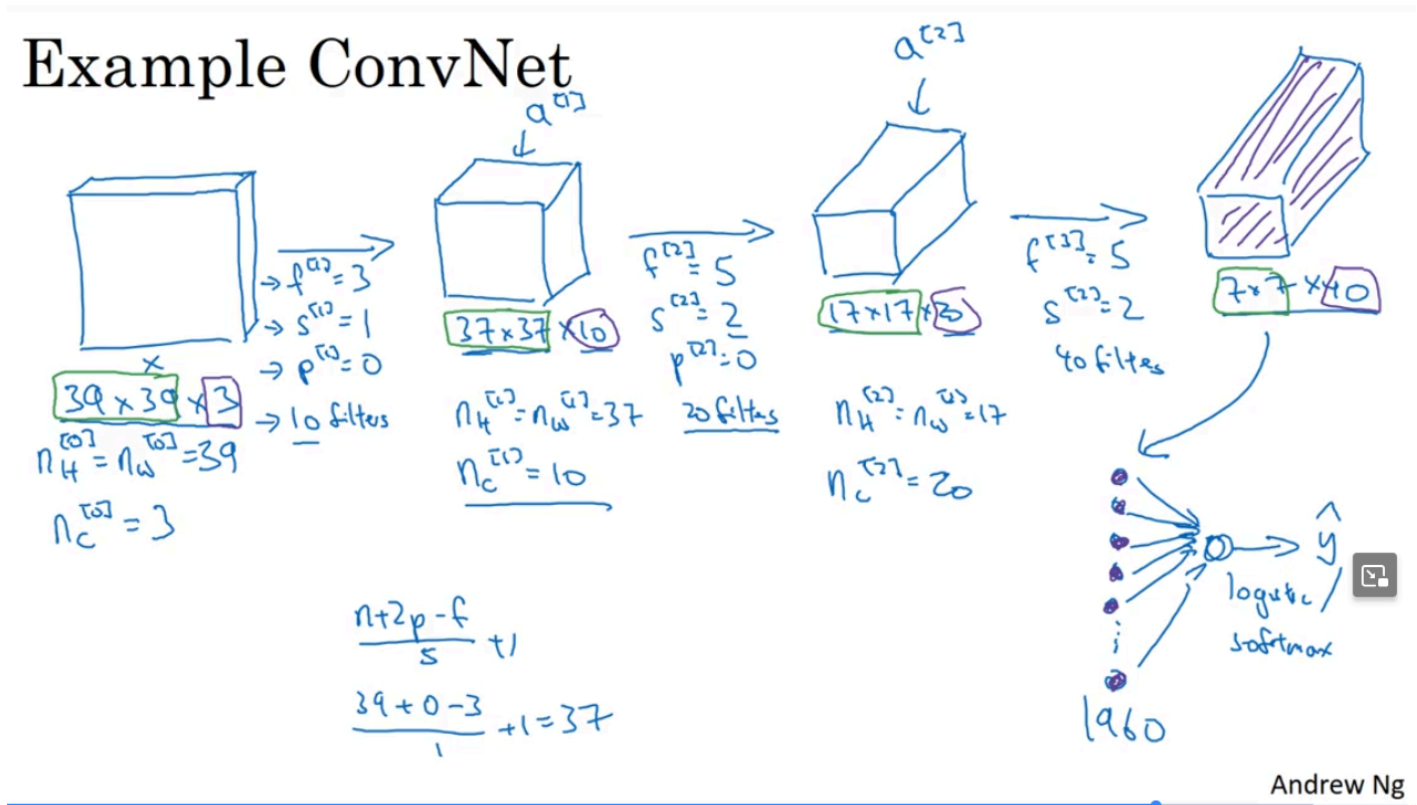
$$n_H^{[l]} \times n_W^{[l]} = \left\lfloor \frac{n_H^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} + 1 \right\rfloor$$

$$A^{[l]} \rightarrow m \times n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$$

Andrew Ng

m: batch size

Simple Convolutional Network Example



This walkthrough demonstrates how individual convolutional layers are stacked to form a **Deep ConvNet**. The goal is to transform a high-resolution image into a compact set of features that can be used for classification.

ConvNet Architecture: A Concrete Example

In this example, we trace an image through three convolutional layers and a final classification step.

Layer-by-Layer Transformation

Layer	Input Size	Filter (f)	Stride (s)	Padding (p)	# Filters	Output Size
Input	39 × 39 × 3	—	—	—	—	—
Conv 1	39 × 39 × 3	3 × 3	1	0	10	37 × 37 × 10
Conv 2	37 × 37 × 10	5 × 5	2	0	20	17 × 17 × 20
Conv 3	17 × 17 × 20	5 × 5	2	0	40	7 × 7 × 40

Key Structural Trends

As you move deeper into a typical ConvNet, the volume dimensions change in a very specific pattern:

- **Height and Width (n_H, n_W):** Generally **decrease** (39 → 37 → 17 → 7). Using a stride > 1 makes this shrink even faster.
- **Number of Channels (n_C):** Generally **increase** (3 → 10 → 20 → 40) as the network learns more complex and diverse features.

From Volumes to Predictions

Once the image has been processed into a small, deep volume (in this case 7 × 7 × 40), it must be converted into a format suitable for classification.

1. **Flattening:** The final volume is "unrolled" into a single long vector.
 - Example: $7 \times 7 \times 40 = 1,960$ total units.
2. **Fully Connected / Softmax:** This vector of 1,960 units is fed into a Logistic Regression (for binary choice like "Cat vs. No Cat") or a Softmax unit (for multi-class recognition).
3. **Output:** The network produces a final prediction ($\hat{y}$), deciding the probability of the input image belonging to a specific class.

Important Terms

- **Flattening:** The process of converting a 3D volume into a 1D vector.

- **Hyperparameters:** The choices you make as a designer, such as filter size (f), stride (s), padding (p), and the number of filters.
- **Valid Convolution:** A convolution performed without padding (p=0), which naturally causes the output to shrink.

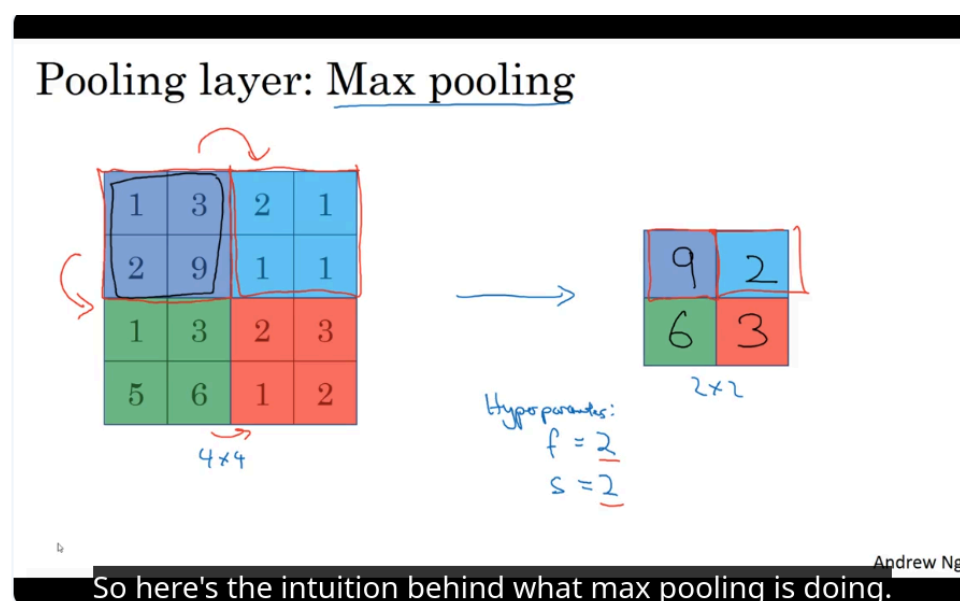
Types of layer in a convolutional network:

- Convolution (conv)
- Pooling (pool)
- Fully connected (FC)



Andrew Ng

Pooling layer



filter_size = 2

stride = 2

Here are your Notion-ready notes, structured for quick scanning and easy review.

🧠 Max Pooling in Convolutional Neural Networks (ConvNets)

🎯 Purpose of Pooling Layers

Pooling layers are added to ConvNets to achieve the following:

- **Reduce representation size:** Shrinks the spatial dimensions of the input.
- **Speed up computation:** Fewer dimensions mean fewer calculations in subsequent layers.
- **Improve robustness:** Makes feature detection more invariant to small translations or shifts in the image.

⚙️ How Max Pooling Works

Max pooling acts like a sliding window over your input data, but instead of doing matrix multiplication, it simply picks the largest number in the window.

Example Scenario:

- **Input:** 4×4 matrix.
- **Output:** 2×2 matrix.
- **Process:** 1. Divide the 4×4 input into four 2×2 regions.
2. Extract the **maximum value** from each region.
3. Place these max values into the corresponding spots of the 2×2 output.

Hyperparameters

Just like convolutional layers, pooling relies on a filter size and a stride:

- **Filter Size (f):** The size of the window (e.g., $f = 2$ means a 2×2 window).
- **Stride (s):** How far the window steps after each operation (e.g., $s = 2$ means it steps over 2 blocks at a time, resulting in no overlap).

💡 The Intuition (Why it works)

If we think of the input grid as a set of activations where a **large number = a detected feature** (e.g., a vertical edge, a cat's eye):

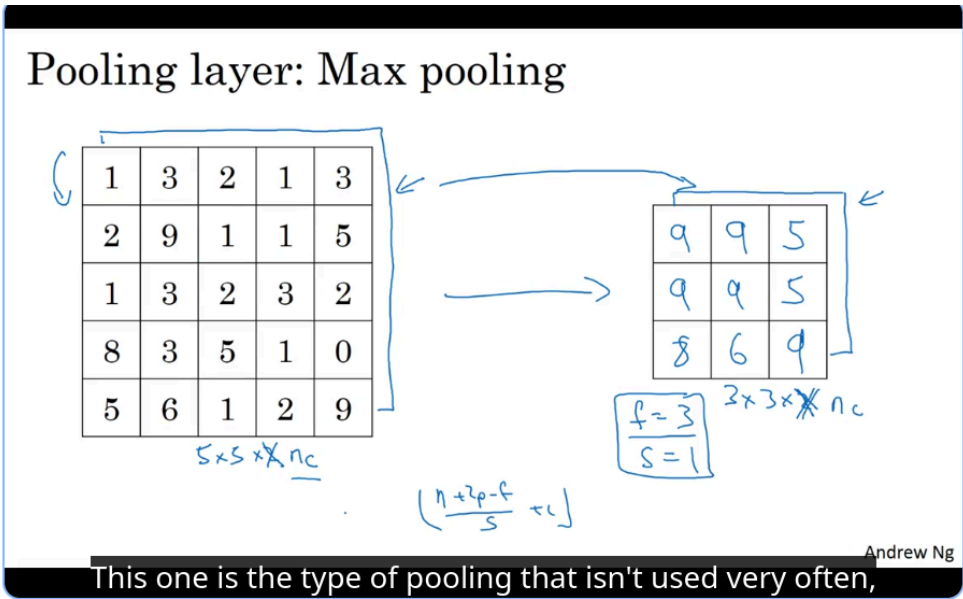
Scenario	Max Pooling Result	Interpretation
Feature exists in the region	Keeps the high number	Preserves the detection of the feature in the output.
Feature does NOT exist	The max value remains small	Accurately reflects the absence of the feature.

Instructor's Note: While the feature-preservation intuition is widely taught, the practical truth is that max pooling is primarily used because **experiments show it works incredibly well** in practice. The exact underlying mathematical reason is still debated!

⚠️ Key Takeaway: No Learnable Parameters

One of the most important properties of Max Pooling is its relationship with gradient descent:

- **Has Hyperparameters:** You must manually set f and s before training.
- **ZERO Parameters to Learn:** There are no weights or biases. It is a fixed, deterministic computation. Gradient descent does not change or learn anything during a max pooling operation.



🔺 Example 2: Overlapping Max Pooling

Max pooling windows can overlap if the stride is smaller than the filter size.

- **Input:** 5×5 matrix
- **Hyperparameters:** Filter size $f = 3$, Stride $s = 1$
- **Output:** 3×3 matrix

How it works with $s = 1$:

1. Place the 3×3 filter over the top-left corner of the 5×5 input and find the max value.

2. Because the stride is 1, shift the filter **one step** to the right. The new 3×3 window will heavily overlap with the previous one. Find the max value.
3. Once the row is complete, shift the window down by **one step** and repeat.



Calculating Output Dimensions

You don't have to guess the output size. The exact same formula used to calculate the output dimensions for standard convolutional layers applies to pooling layers:

$$Output_Dimension = \lfloor \frac{n + 2p - f}{s} + 1 \rfloor$$

- **n** = input dimension size
- **p** = padding (rarely used in max pooling, usually 0)
- **f** = filter size
- **s** = stride

(Testing the formula on the example above: $(5 + 0 - 3) / 1 + 1 = 3$, which perfectly matches our 3×3 output!)

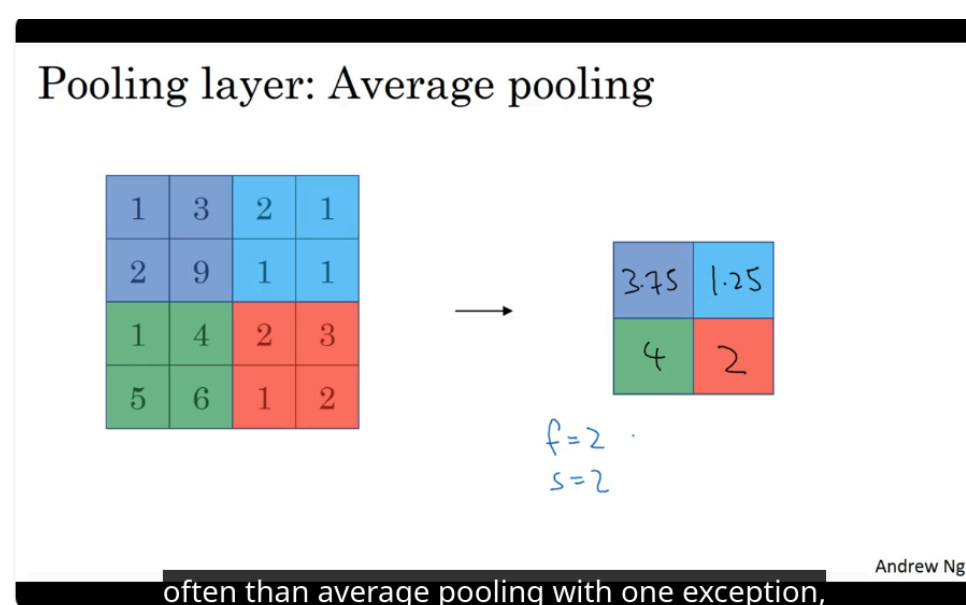


Max Pooling on 3D Inputs (Multiple Channels)

Images and feature maps in ConvNets rarely have just one channel (2D). They usually have multiple channels (3D), like RGB color channels or multiple feature detectors.

- **Key Rule:** Max pooling **preserves the number of channels**. It only shrinks the height and width (n_H and n_W).
- **Example:** If you apply max pooling to a $5 \times 5 \times 2$ input (using $f = 3$, $s = 1$), your output will be $3 \times 3 \times 2$.
- **The Process:** The max pooling computation is performed **independently** on each channel.
 - Slice 1 (Channel 1) gets pooled to create Output Slice 1.
 - Slice 2 (Channel 2) gets pooled to create Output Slice 2.
 - *Generalizing:* An input with N_C channels will always output N_C channels.

Average Pooling



Average Pooling

While max pooling is the dominant method, there is another variation called **Average Pooling**.

Instead of picking the largest number in the sliding window, average pooling calculates the **mean (average)** of all the numbers within that filter region.

Example Scenario:

- **Input:** 4×4 matrix.
- **Hyperparameters:** Filter size $f = 2$, Stride $s = 2$.

- **Process:** The 4×4 grid is broken into four distinct 2×2 regions (because stride = 2). You calculate the average of the 4 numbers in each region, resulting in a 2×2 output (e.g., values like 3.75, 1.25, 4.0, 2.0).

Max Pooling vs. Average Pooling (When to use which?)

As a general rule in modern Deep Learning: **Max Pooling > Average Pooling**. Max pooling is used significantly more often in standard convolutional layers.

However, there is **one major exception** where average pooling is highly useful:

Collapsing Representations (Global Average Pooling)

Average pooling is often used very deep in the neural network (usually right before the final output layers) to completely collapse the spatial dimensions of your data.

- **The Goal:** Take a feature map with spatial width and height, and average those dimensions out so you are only left with the channel depths.
- **Example:** You have an input representation of $7 \times 7 \times 1000$.
- **The Operation:** You take an average over the entire 7×7 spatial grid.
- **The Result:** The output collapses down to $1 \times 1 \times 1000$.

(Note: You will see practical examples of this specific architecture later in the Coursera course!)

Summary of pooling

Hyperparameters:

- f : filter size
- s : stride
- Max or average pooling

~~p: padding~~

No parameters to learn!

$$n_H \times n_W \times n_C$$

$$\downarrow$$

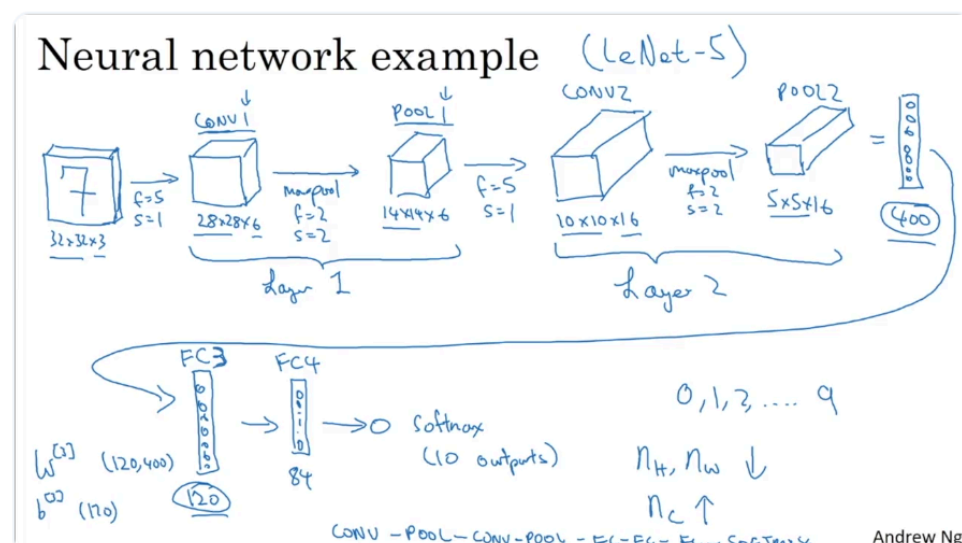
$$\left\lfloor \frac{n_H - f + 1}{s} \right\rfloor \times \left\lfloor \frac{n_W - f + 1}{s} \right\rfloor \times n_C$$

Andrew Ng

And then beyond that, you are done.

When apply Max Pooling, do not use padding!

CNN Example



Here is the next section of your notes covering a full ConvNet architecture, ready for your Notion workspace!

Building a Full ConvNet: A Practical Example

This architecture is inspired by the classic **LeNet-5** (created by Yann LeCun), originally designed for recognizing handwritten digits (0-9).

1. The Architecture Walkthrough

Stage	Type	Hyperparameters	Output Shape	Notes
Input	Image	-	$32 \times 32 \times 3$	Standard RGB image of a digit.
Layer 1	Conv 1	$f=5, s=1, p=0$, 6 filters	$28 \times 28 \times 6$	Uses formula: $(n + 2p - f)/s + 1$
	Pool 1	Max Pool, $f=2, s=2$	$14 \times 14 \times 6$	Halves spatial dimensions.
Layer 2	Conv 2	$f=5, s=1, p=0$, 16 filters	$10 \times 10 \times 16$	Number of channels (n_C) increases.
	Pool 2	Max Pool, $f=2, s=2$	$5 \times 5 \times 16$	Flattening this gives 400 units.
Layer 3	FC 3	120 units	120×1	Fully connected. Weight matrix (W_3) is 120×400 .
Layer 4	FC 4	84 units	84×1	Fully connected layer.
Output	Softmax	10 units	10×1	Outputs probabilities for digits 0-9.

Layer Counting Convention

When reading literature or discussing ConvNets, you'll encounter two ways to count layers. In this course, we use the **Weight-Based Convention**:

- **The Rule:** A "layer" is defined as a component that has **parameters (weights) to learn**.
- **The Result:** Because pooling layers have no weights (only hyperparameters), they are grouped together with the preceding convolutional layer.
 - *Example:* The **Conv 1** and **Pool 1** operations together make up **Layer 1**.

General Trends & Patterns

As you go deeper into a standard Convolutional Neural Network, you will typically observe the following trends:

1. **Spatial Dimensions Shrink:** The height (n_H) and width (n_W) will steadily decrease (e.g., $32 \rightarrow 28 \rightarrow 14 \rightarrow 10 \rightarrow 5$).
2. **Depth Increases:** The number of channels (n_C) will grow as the network detects more complex, high-level features (e.g., $3 \rightarrow 6 \rightarrow 16$).
3. **The Common Pattern:** Most traditional networks follow a specific rhythm:

[CONV -> POOL] -> [CONV -> POOL] -> [FC] -> [FC] -> Softmax

Pro Tip for Hyperparameters

Choosing filter sizes, strides, padding, and the number of layers can feel overwhelming.

- **The Guideline:** Do not try to invent your own hyperparameter settings from scratch.
- **The Solution:** Look at the literature! Find an architecture that has already proven successful for others (like LeNet, AlexNet, or VGG), and adapt it to your specific application.

Why Convolutions?

Starting around 2:15 minute, the number of parameters should have been:

$(5 \times 5 \times 3 + 1) \times 6 = 456$

This is based on the equation:

$(f^{[l]} \times f^{[l]} \times n_c^{[l-1]} + 1) \times n_c^{[l]}$

$f^{[l]}$ is the filter height (and width).

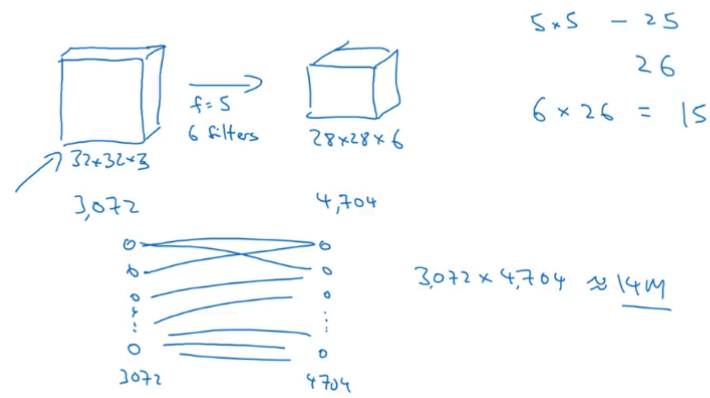
$n_c^{[l-1]}$ is the number of channels in the previous layer.

$n_c^{[l]}$ is the number of channels in the current layer.

The "1" is the bias term.

(It was $(5 \times 5 + 1) \times 6 = 156$ in the video.)

Why convolutions



Andrew Ng

which is equal to 156 parameters.

Here is the next section for your Notion notes, focusing on the core advantages of Convolutional Neural Networks!

Why Convolutions? (The Power of ConvNets)

When processing images, standard Fully Connected (FC) neural networks quickly become inefficient. Convolutional layers offer two massive advantages over traditional dense layers:

1. **Parameter Sharing:** A feature detector (like a vertical edge detector) that is useful in one part of the image is likely useful in another part of the image.
2. **Sparsity of Connections:** In each layer, each output value depends only on a small number of inputs (just the pixels within the filter's window), rather than every single pixel in the entire image.

The Parameter Advantage: A Mathematical Example

To truly understand why ConvNets are so powerful, let's compare the number of parameters needed for a standard Fully Connected network versus a Convolutional network using the same input and output dimensions.

The Scenario:

- **Input Image:** $32 \times 32 \times 3$ (RGB image) $\rightarrow$ Total inputs = 3,072 units
- **Desired Output:** $28 \times 28 \times 6 \rightarrow$ Total outputs = 4,704 units

Approach 1: Fully Connected Layer

If we were to connect every single input unit to every single output unit (like a standard neural network):

- **Weight Matrix Calculation:** $3,072 \times 4,704$
- **Total Parameters:** ≈ 14.4 million parameters

Note: This is an incredibly high number of parameters to learn for just a tiny 32×32 image. If the image were larger (e.g., 1000×1000), the number of parameters would become computationally impossible to train.

Approach 2: Convolutional Layer

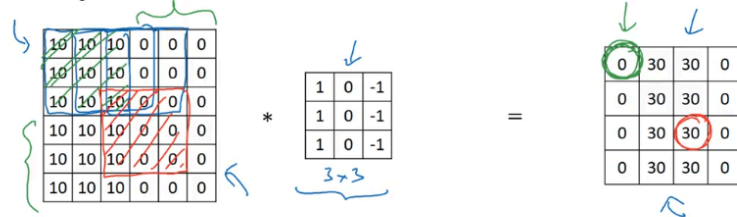
Instead of connecting everything, we use **6 filters**, each sized **5×5** (with a stride of 1).

- **Parameters per filter:** $5 \times 5 = 25$ weights + 1 bias = 26 parameters per filter.
- **Total parameters for all filters:** 26 parameters $\times$ 6 filters
- **Total Parameters:** 156 parameters

The Verdict

By using convolutional layers, we drastically reduced the number of parameters to learn from **~ 14 million down to just 156**. This prevents overfitting, requires significantly less memory, and makes training deep networks computationally feasible.

Why convolutions



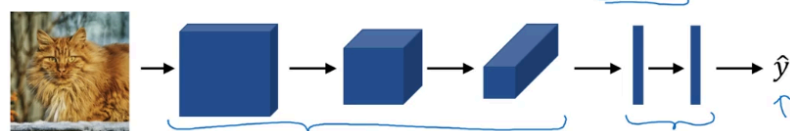
Parameter sharing: A feature detector (such as a vertical edge detector) that's useful in one part of the image is probably useful in another part of the image.

→ **Sparsity of connections:** In each layer, each output value depends only on a small number of inputs.

Andrew Ng

Putting it together

Training set $(x^{(1)}, y^{(1)}) \dots (x^{(m)}, y^{(m)})$.



$$\text{Cost } J = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

Use gradient descent to optimize parameters to reduce J

Andrew Ng

🧠 Deep Dive: Why Convolutions Work So Well

As we saw in the previous example, ConvNets require drastically fewer parameters than Fully Connected networks. This efficiency comes down to two primary concepts: **Parameter Sharing** and **Sparsity of Connections**, which together also create **Translation Invariance**.

1. Parameter Sharing

The Core Idea: A feature detector that is useful in one part of the image is probably useful in another part of the image.

- **How it works:** If the network learns a 3×3 filter to detect a vertical edge in the upper-left corner, it doesn't need to learn a brand-new set of weights to detect a vertical edge in the lower-right corner. It just slides (shares) that exact same filter across the entire image.
- **What it applies to:** This intuition holds true for both **low-level features** (like edges or color blobs) and **high-level features** (like a cat's eye or a human face).
- **The Benefit:** Instead of learning separate detectors for every spatial location, sharing a single set of parameters across the whole image massively reduces the computational load.

2. Sparsity of Connections

The Core Idea: In any given layer, each output value depends only on a tiny fraction of the input values.

- **How it works:** If you are using a 3×3 convolutional filter, a single pixel in the output feature map is calculated using *only* 9 specific pixels from the input image.
- **The Disconnect:** The other hundreds or thousands of pixels in the input image have **absolutely zero effect** on that specific output pixel.
- **The Benefit:** Unlike a Fully Connected layer where every output relies on every single input, sparse connections keep the math localized, further reducing the number of weights the network has to memorize.

The Ultimate Result: Translation Invariance & Robustness

Because ConvNets rely on parameter sharing and sparse connections, they naturally develop a property called **Translation Invariance**.

- **What is it?** A picture of a cat shifted a few pixels to the right is still a picture of a cat. A good neural network shouldn't get confused just because the subject moved slightly.
- **Why ConvNets excel at this:** Because the network applies the exact same filter across all positions of the image (in both early and late layers), it automatically learns to recognize features regardless of where they appear.
- **Final Advantage:** Because there are fewer parameters to train overall, ConvNets can be trained effectively even with smaller datasets and are **much less prone to overfitting**.

Up Next: Now that we understand the building blocks and the theory, the final step is putting it all together to see how we actually train a Convolutional Neural Network with a labeled training set!